

RAG BASED COMPLIANCE ASSISTANT

A cost-efficient Retrieval-Augmented Generation chatbot that leverages context compression to minimize unnecessary tokens sent to the LLM.

Our compression pipeline reduces inference costs and response latency while maintaining accurate, citation-backed answers for complex financial regulations.

PRESENTED BY :

PRESENTED TO :

RAG
HOW

Retrieval

RETRIEVAL

DATA INGESTION & RETRIEVAL ARCHITECTURE

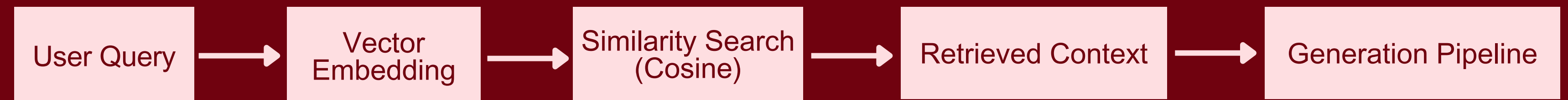
Our objective is to establish a robust pipeline to process dense, highly structured compliance and regulatory documents for accurate LLM querying.

INGESTION PHASE



- Transforms unstructured regulatory documents into optimized, searchable vector embeddings.

RETRIEVAL PHASE



- Translates user queries into vector searches to pinpoint and deliver exact regulatory context.

EVALUATING DIFFERENT CHUNKING STRATEGIES

We actively implemented and tested three baseline chunking methodologies, such as Fixed-Size Token, Recursive Character, and Semantic Chunking strategies in order to evaluate their effectiveness on our regulatory documents.

OBSERVED LIMITATIONS

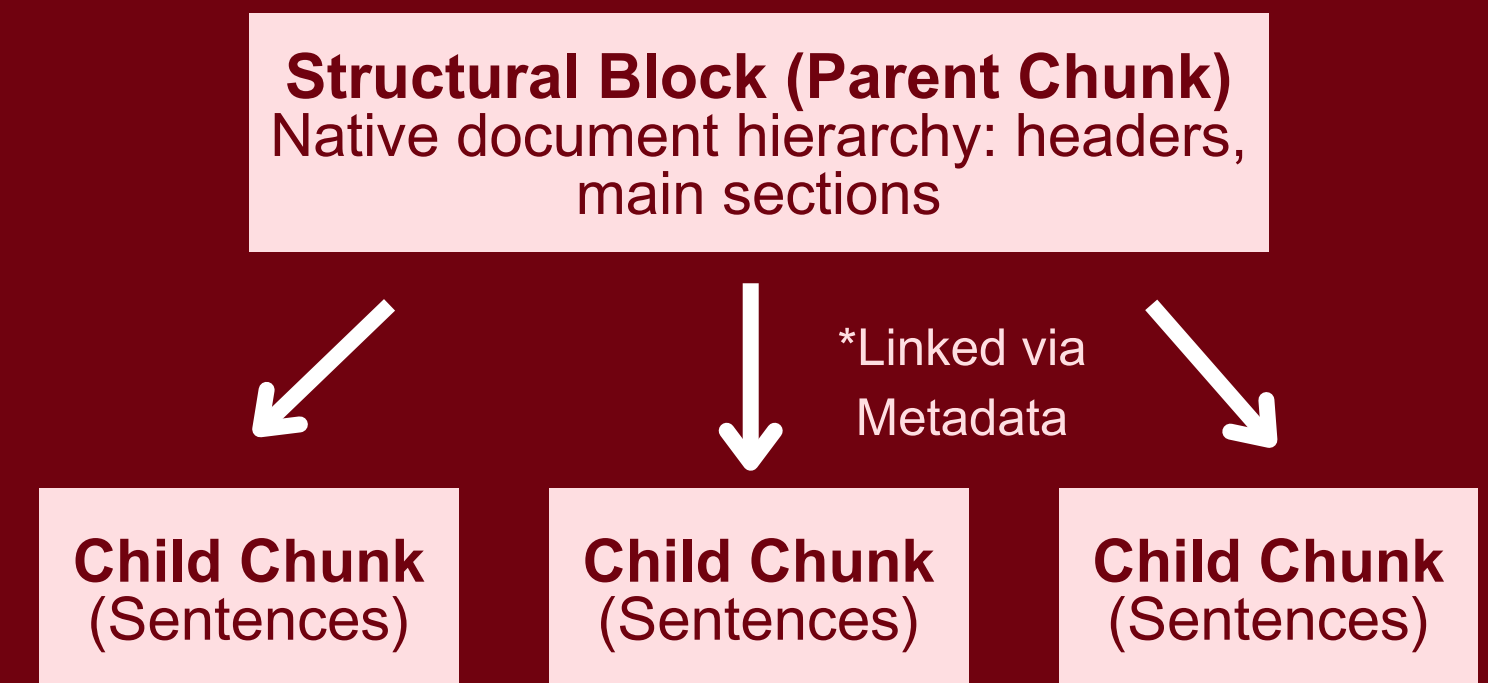
Fixed Size & Recursive

Caused severe context fragmentation; split critical legal clauses and exceptions.

Semantic Chunking

Computationally heavy during ingestion; inaccurately merged distinct rules together.

MIXED ARCHITECTURE



The Result : We proceeded with this Structural + Parent-Child combination because it yielded significantly higher retrieval quality compared to the other strategies, ensuring the LLM always receives highly precise hits backed by full legal context.

HIGH-FIDELITY CONTEXT RETRIEVAL

THE MECHANISM

- User queries are converted into vector embeddings and matched strictly against the small, specific "Child" chunks (sentences/sub-clauses) stored in the Vector Database.
- Upon finding a high-similarity match, the system uses metadata links to pull the corresponding "Parent" chunk which is the complete structural section.

THE ADVANTAGE

- The LLM gets the exact pinpointed answer from the child chunk without losing the regulatory scope, definitions, or exceptions from the parent.
- By grounding the generation model in complete structural context, the accuracy of compliance answers drastically improved, hence reducing hallucinations.

PERFORMANCE METRIC : SOURCE FIT-RATE

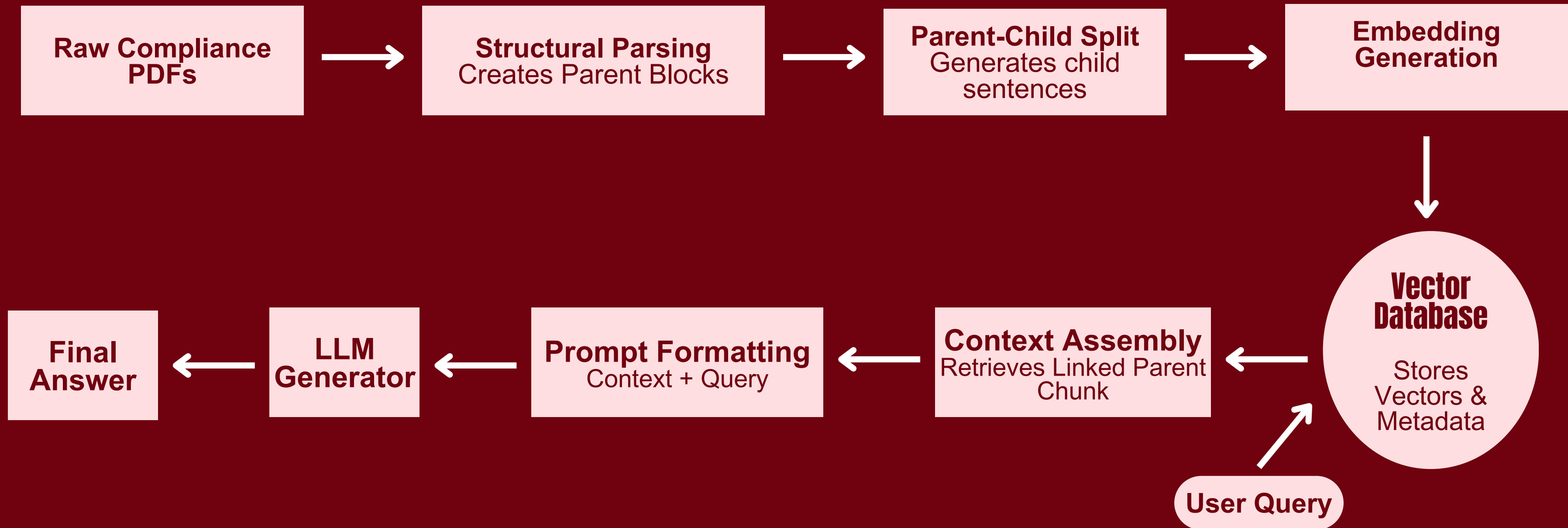
Measures the density of highly relevant chunks within the retrieved context using the Gemini LLM with a structured prompt to evaluate and score the semantic alignment between the user's query and the retrieved chunks.



Achieved a rate peaking >80%, proving that the retrieved context directly and accurately supported the user's queries.

BASELINE PIPELINE

THE INITIAL END-TO-END DATA FLOW USING THE STRUCTURAL + PARENT-CHILD CHUNKING APPROACH.



While this pipeline successfully retrieved high-quality context, we needed to optimize token costs before generation. However, integrating standard context compression introduced unexpected bottlenecks.

Generation

GENERATION

CHALLENGES IDENTIFIED

AFTER IMPLEMENTING THE GENERATION PIPELINE USING HEADROOM KOMPRESS, THE SYSTEM PRODUCED GOOD RESPONSES. HOWEVER, PERFORMANCE TESTING REVEALED TWO MAJOR CHALLENGES.

1. HIGH COMPRESSION LATENCY

Headroom compression introduced an additional 12–15 seconds of processing time per query while achieving only ~25% token reduction. The limited compression benefits did not justify the significant increase in response latency, making it unsuitable for deployment in a real-time chatbot where fast response times are critical.

2. LOW CONTEXT QUALITY

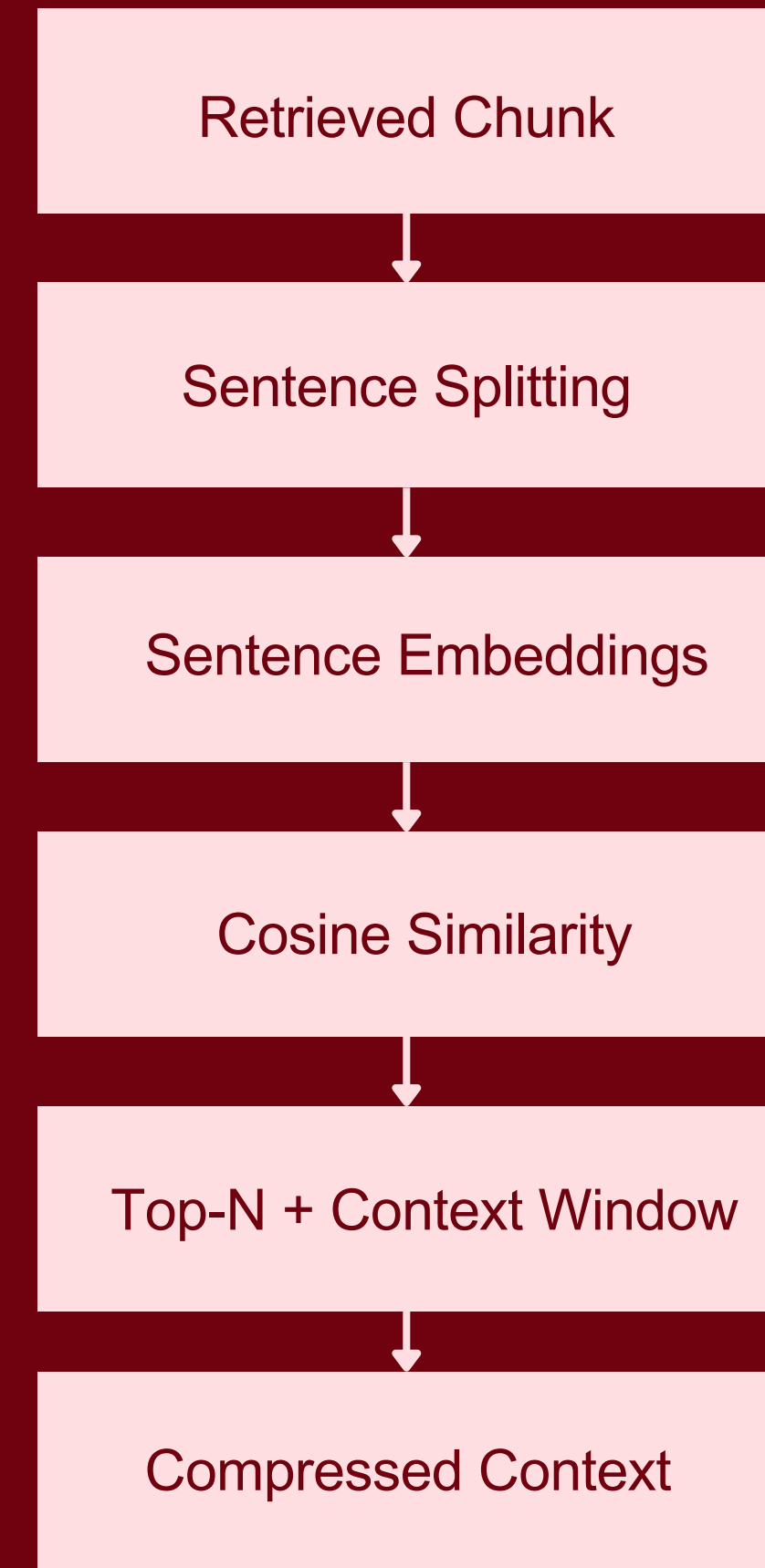
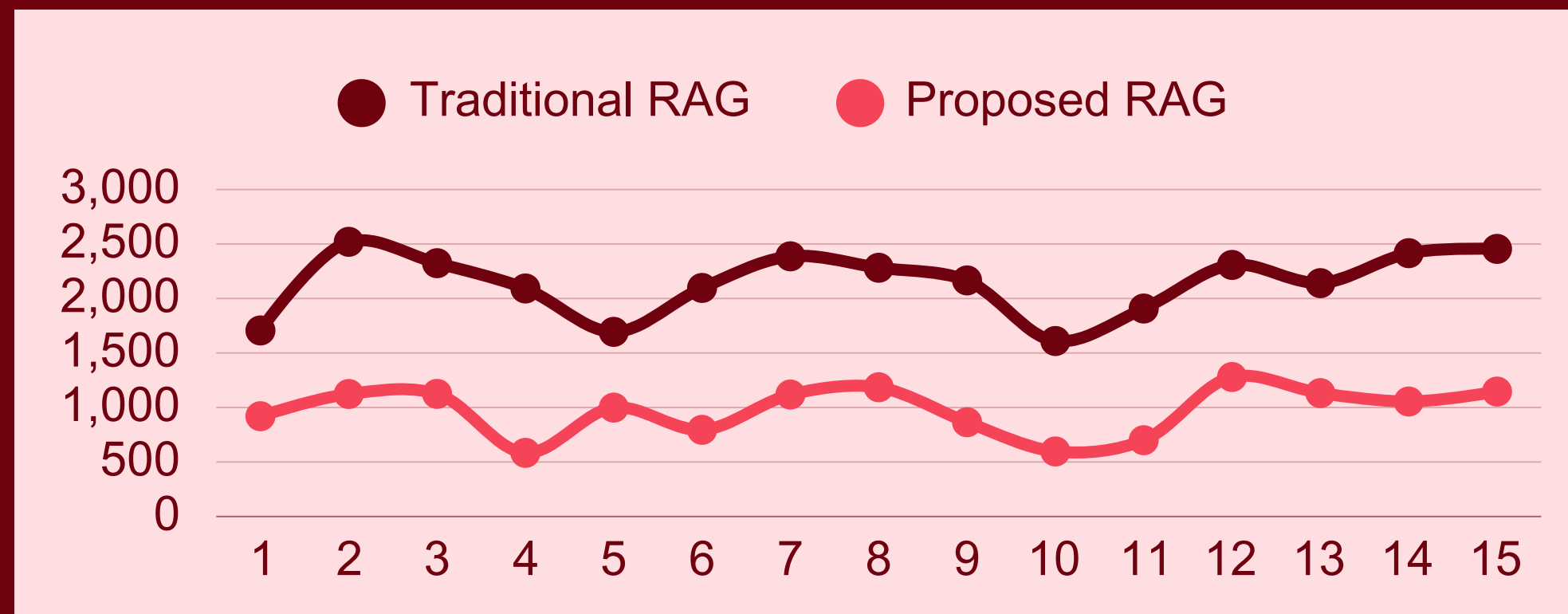
The Context Recall was approximately 0.60, indicating that relevant information was occasionally omitted during compression. This resulted in incomplete context being passed to the language model, reducing the completeness and reliability of the generated responses.

SENTENCE-AWARE COMPRESSION

TO OVERCOME THE HIGH LATENCY OF HEADROOM KOMPRESS, WE DESIGNED A LIGHTWEIGHT SENTENCE-AWARE COMPRESSION STRATEGY.

Working Principle

- Split each retrieved document chunk into individual sentences.
- Generate embeddings for each sentence.
- Compute cosine similarity between the user query and every sentence.
- Select the Top-N most relevant sentences.
- Include neighboring sentences using a context window to preserve coherence.
- Merge the selected sentences and pass the compressed context to the LLM.

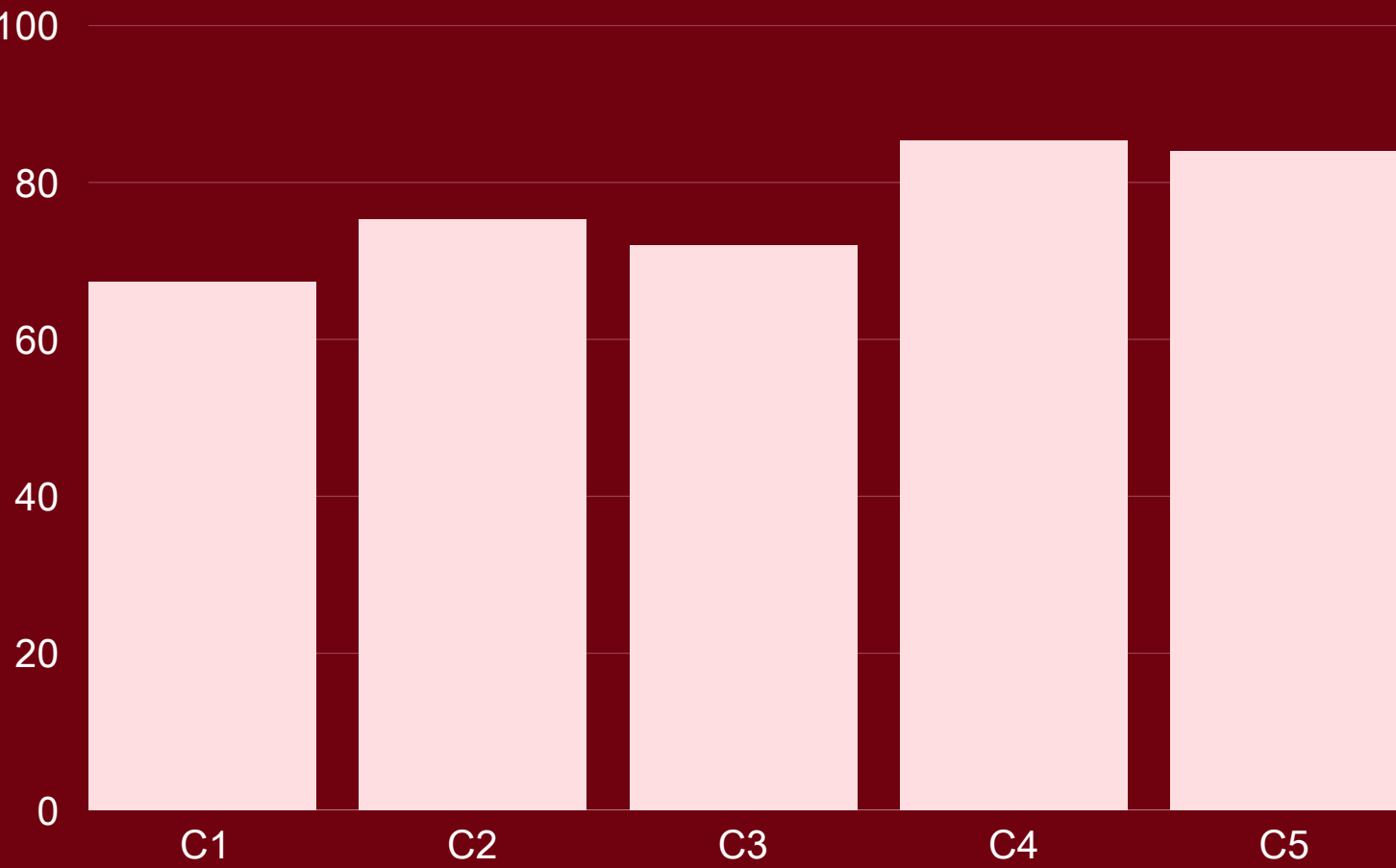


EMBEDDING MODEL EVALUATION

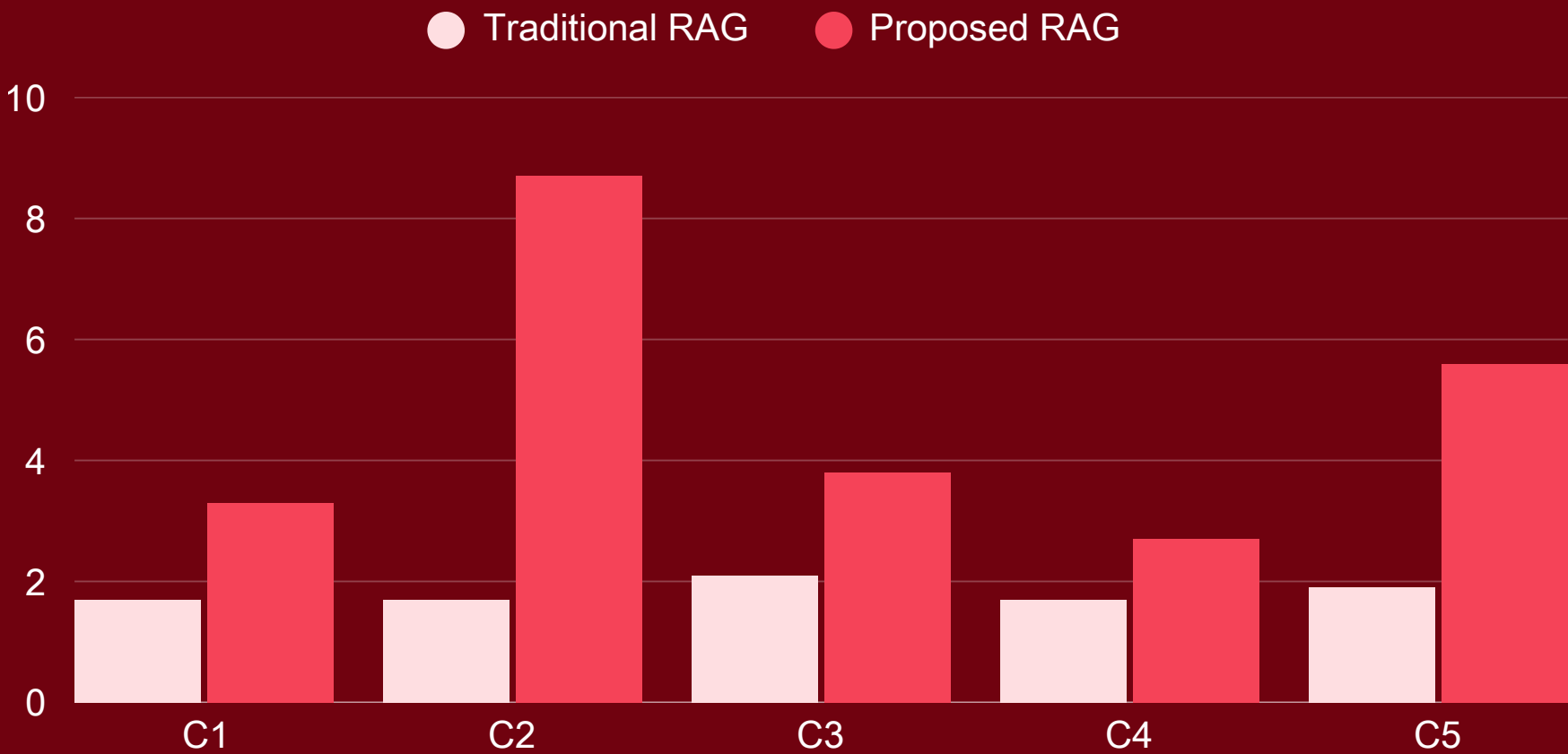
Since sentence embeddings are the core of the compression pipeline, the choice of embedding model directly affects both latency and context recall.

EMBEDDING MODEL CANDIDATES

- MiniLM-L6-v2 (Dimension: 384, Size: ~80 MB)
- BGE Small v1.5 (Dimension: 384, Size: ~130 MB)
- BGE Base v1.5 (Dimension: 768, Size: ~438 MB)



Combination No.	Ingestion & Retrieval Embedding Model	Sentence Compression Embedding Model
C1	Mini-LM	Mini-LM
C2	BGE Small	BGE Base
C3	BGE Small	BGE Small
C4	BGE Base	BGE Small
C5	BGE Base	BGE Base



RUNTIME OPTIMIZATION

CHALLENGE

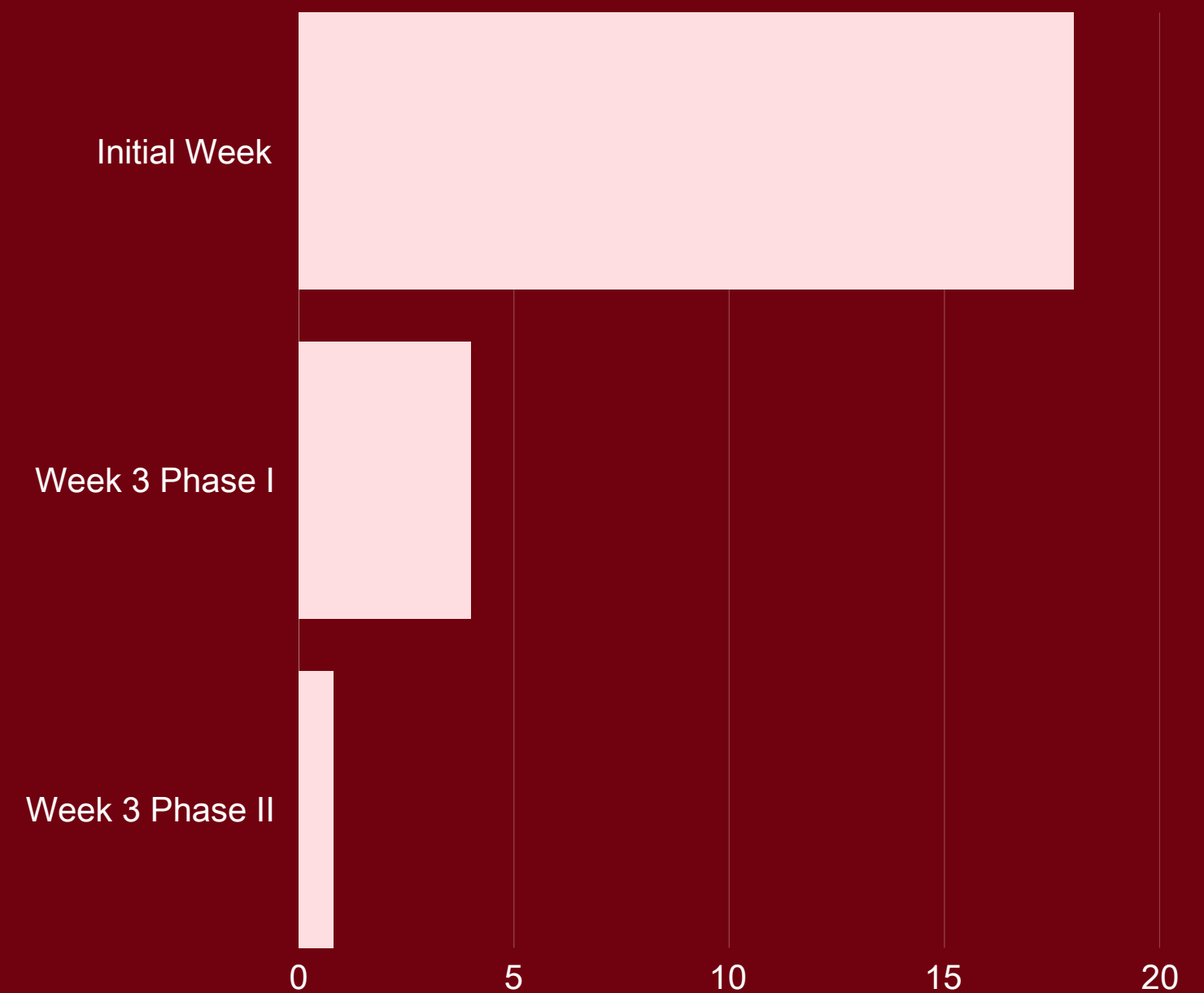
- After integrating the frontend, the overall response time increased to 4–5 seconds.
- On analysis, we found that generating embeddings for every query at runtime was causing the additional delay and also the gemini LLM took 1.5 to 2.5 seconds for a each query

SOLUTION

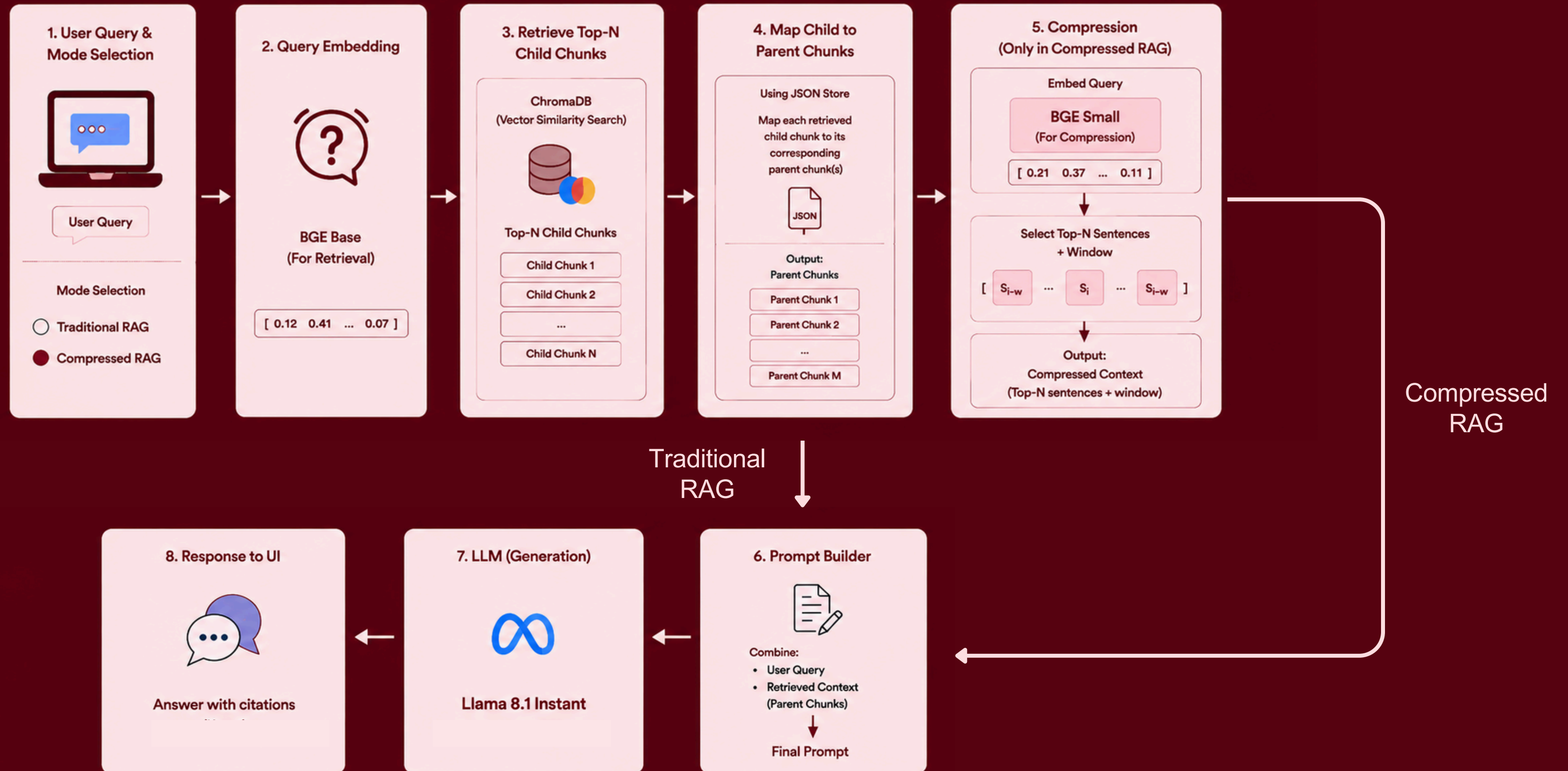
- We shifted the embedding generation to the document ingestion phase.
- During runtime, the system simply loads the precomputed embeddings instead of generating them again.
- And we choose llama 8.1 instant which does the same as gemini but in just 0.6 seconds average

RESULT

- Reduced the end-to-end latency to ~0.8 seconds.
- Achieved faster responses without compromising retrieval quality.



ARCHITECTURE



THANK YOU

THANK YOU